

FedProp: Federated Graph Neural Networks with Feature Propagation

Anonymous submission

Abstract

Graph Neural Networks (GNNs) are a powerful tool to learn from relational data; however, training them in federated, privacy-preserving settings still poses considerable challenges. A key obstacle is the presence of so-called cross-client edges, i.e., edges whose incident nodes reside on distinct devices. Because standard message-passing GNNs aggregate features across neighboring nodes, these cross-client edges lead to information gaps that degrade accuracy when communication between client devices is disallowed. We recast this obstacle as a missing-feature problem and introduce FedProp: a model-agnostic federated GNN framework that reconstructs the 1-hop features required for aggregation via feature propagation. In contrast to prior approaches, which either learn additional models to generate the missing neighbors or communicate the missing information, FedProp offers a lightweight, iterative process that can be executed locally without requiring inter-client communication or additional learnable parameters. First, we provide a theoretical justification for feature propagation in subgraphs and provide a convergence guarantee of the iterative process. We then empirically demonstrate that FedProp can significantly improve GNN performance. For example, on Cora with a GCN backbone, FedProp achieves 80.8% accuracy, improving by 17% over the federated baseline and reaching 99.6% of the performance of the centralized model. FedProp thus enables users to deploy powerful GNNs across various client devices with locally stored data, achieving near-optimal accuracy and minimal communication costs.

1 Introduction

Graph Neural Networks (GNNs) are powerful tools for learning from graph-structured data (Khemani et al. 2024). Operating via a message-passing mechanism where nodes aggregate local neighborhood information to richer representations (Fu et al. 2022), GNNs have achieved impressive performance across diverse applications, including social networks, drug discovery, and recommendation systems (Liu et al. 2022).

However, training GNNs on real-world graphs often faces a critical challenge: data is frequently decentralized and subject to privacy regulations like GDPR, necessitating a move towards frameworks like Federated Learning (FL) (Peng et al. 2023; Li et al. 2023). This paper addresses a practical and increasingly common setting known as **'Subgraph-FL'**

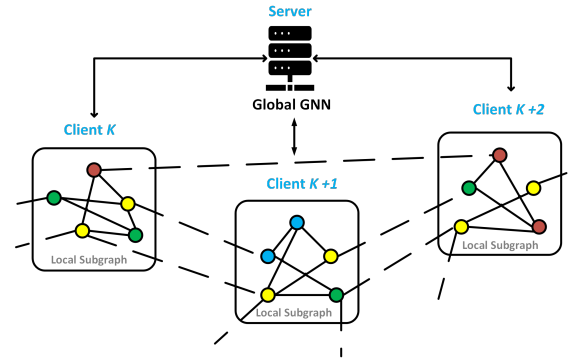


Figure 1: Cross-client edges (dashed lines) connect nodes on distinct clients, leading to missing neighbor information that degrades GNN message passing.

(?), where clients collaboratively train a model on their local subgraphs, which are partitions of a single, larger global graph. The 'Subgraph-FL' setting is applicable in a range of applications. In financial fraud detection, for instance, a transaction graph may be partitioned across different banks, yet a fraudulent transaction ring could span multiple institutions. Similarly, in recommendation systems, user-item interaction graphs might be partitioned by region, but user preferences can have global patterns.

In this setting, the use of graph neural networks becomes quite challenging. This is due to the fact that (i) each client only sees a subset of the nodes of the underlying graph and (ii) clients are only allowed to exchange model updates (Sun et al. 2025; Cui et al. 2025) without taking into consideration the cross-client edges that connect graph nodes in distinct client devices (see Figure 1).

Such cross-client edges occur frequently in large classes of partitioned graphs (e.g. social networks or transportation networks). FL solutions that ignore them are prone to lose key information with consequent degradation of overall GNN performance, particularly in semi-supervised node classification. For example, when compared to a centralized solution (Yao et al. 2023), local training on Ogbn-Arxiv and Cora without accounting for cross-client connectivity resulted in a 14.6% and a 19.4% drop in accuracy, respectively. It is conceivable that clients could generate synthetic

neighbors for their subgraphs or share partial information with other clients. However, those approaches incur significant communication overhead, require additional learnable parameters, or fail at ensuring the necessary level of privacy. This motivates the need for more effective federated GNN methodologies.

The goal of this paper is to overcome the aforementioned challenges by enabling clients to reconstruct missing neighbor information locally and efficiently without exchanging raw features or learned representations. Our main contributions are as follows:

1. **Federated GNN framework:** We present FedProp, a model-agnostic federated GNN framework that solves the missing neighbor problem by recasting it as feature imputation via efficient iterative propagation within local subgraphs.
2. **Theoretical Analysis:** We offer mathematical proof that FedProp’s iterative propagation process converges as desired.
3. **Enhancement and Empirical Validation:** We enhance FedProp with lightweight positional encodings, which improve its accuracy. Empirical results across different datasets show that FedProp achieves 97–99% of the accuracy of the reference centralized system.

The remainder of the paper is organized as follows. Section 2 reviews related work, Section 3 provides fundamental background, and Section 4 presents the proposed FedProp methodology, including problem definition, algorithm, and convergence analysis. Section 5 discusses the experimental setup and presents the results. The paper ends with some conclusions, summarized in Section 6.

2 Related Work

Federated learning is a collaborative machine learning method that trains models across devices or data silos without sharing private data, thereby ensuring privacy and security (Banabilah et al. 2022). A key algorithm is Federated Average (FedAvg) (McMahan et al. 2023), which averages locally trained models to update a global model. FedAvg sometimes struggles with non independently and identically distributed (non-iid) data (Li et al. 2020). To address data heterogeneity, FedProx improves on FedAvg by adding a regularization term for more stable local updates (Li et al. 2020). Another notable algorithm, FedAdam, adapts the Adam optimizer for federated settings, providing faster and more stable convergence with variable client data distributions (Reddi et al. 2021). These algorithms enable the training of decentralized and privacy-preserving models in various applications (Silva, Vinagre, and Gama 2023).

GNNs are a class of machine learning models designed to learn from graph-structured data by aggregating information from neighboring nodes in the underlying graph. Popular GNNs models include GCN (Kipf and Welling 2017), GAT (Veličković et al. 2018), and GraphSAGE (Hamilton, Ying, and Leskovec 2018). In federated learning settings, where data is distributed across multiple clients and cannot be centralized due to privacy or regulatory restrictions, neighborhood aggregation becomes a significant challenge

(He et al. 2021). As a result, the best-performing GNN models in centralized environments may not perform equally well in federated scenarios.

Federated GNN methods address the missing neighbor problem in partitioned graphs via two main strategies. *Generative approaches*, such as FedSage+ (Zhang et al. 2021) and FedNI (Peng et al. 2023), use local generative models (e.g., GANs) to impute missing neighbors or features, but introduce significant computational and communication overhead. *Transmission-based approaches*, including FedGCN (Yao et al. 2023) and FedGAT (Ambekar et al. 2024), share required neighbor features in a pre-training round and employ polynomial approximations (for GAT). However, they require extra communication and encryption, while being limited to specific GNN architectures. (Add a Note on FedCog)

FedProp differs fundamentally from the aforementioned methodologies in several ways: (a) FedProp is model-agnostic, (b) requires no extra inter-client communication, and (c) reconstructs missing features locally via feature propagation. Unlike other federated GNN methods, such as FedGCN, FedGAT, and FedSage+, which face significant trade-offs in privacy, communication overhead, and computational cost, FedProp fully preserves privacy with low communication, while maintaining broad compatibility with any existing message-passing GNN.

3 Preliminaries

Graph Neural Networks (GNNs) learn node embeddings by iteratively aggregating features from local neighborhoods. A standard GNN update rule at layer ℓ is

$$h_v^{(\ell+1)} = f(h_v^{(\ell)}, \{h_u^{(\ell)} : u \in N(v)\}, x_v),$$

where f is a learnable aggregation function and $N(v)$ are neighbors of v and x_v is the feature vector of the node v . The final representation $h_v^{(L)}$ captures information up to L hops.

In a federated setup, the graph G is partitioned across K clients with subgraphs G_k . Each node $v \in V_k$ has known neighbors $N_K(v) = N(v) \cap V_k$ and unknown neighbors $N_U(v) = N(v) \setminus V_k$ (connected through cross-client edges). Without cross-client communication, GNNs on client k can only aggregate from $N_K(v)$, leading to significant performance degradation. FedProp addresses this by reconstructing initial features for unknown neighbors, allowing GNNs to leverage full neighborhood information so that the new GNN update rule becomes:

$$h_v^{(\ell+1)} = f\left(h_v^{(\ell)}, \{h_u^{(\ell)} : u \in N_K(v)\}, \{\tilde{h}_u^{(\ell)} : u \in N_U(v)\}, x_v\right).$$

Node feature propagation is a process that diffuses information across a graph, smoothing node features by iteratively aggregating information from local neighborhoods via a diffusion operator $\hat{P}$: $X^{(t+1)} = \hat{P} X^{(t)}$, where $X^{(0)} = X$. This process is key for imputing missing node features: by repeatedly applying $\hat{P}$ to a partially observed feature matrix, known information spreads to fill in unknown values based on graph connectivity and smoothness (Rossi et al. 2022).

Standard diffusion operators include symmetric normalized adjacency, Personalized PageRank (Gasteiger, Bojchevski, and Günnemann 2022), and Heat Kernel, with details provided in Appendix A.2.

Dirichlet energy quantifies the smoothness of a signal on a graph, with low energy indicating similar features across neighboring nodes. For a node-feature matrix X and the symmetric normalized Laplacian $L = I - D^{-1/2}AD^{-1/2}$, the Dirichlet energy is defined as:

$$\mathcal{E}(X) = \text{tr}(X^\top LX) = \frac{1}{2} \sum_{(i,j) \in E} A_{ij} \left\| \tilde{X}_i - \tilde{X}_j \right\|_2^2, \quad (1)$$

where $\tilde{X}_i = X_i / \sqrt{1 + D_{ii}}$ are normalized features. This energy is crucial for monitoring smoothness in GNNs (Zhou et al. 2021) and guides imputation: feature propagation minimizes $\mathcal{E}(X)$ subject to known boundary constraints, effectively filling in missing values (Rossi et al. 2022).

4 The FedProp Method

As established in the Introduction, we address the challenge of federated learning in the ‘**Subgraph-FL**’ setting. In this scenario, a global graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is divided between K clients, where each client k has a local subgraph $\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k)$ and its corresponding node features $\mathbf{X}_k$. The core challenge comes from the message-passing mechanism of GNNs. For a node $v \in \mathcal{V}_k$ to compute its updated representation, it must aggregate information from its entire neighborhood, $\mathcal{N}(v)$. However, in a standard federated setup, client k only possesses features for nodes within its own partition, $\mathcal{V}_k$. It has no access to the features of any remote neighbors—that is, any node $u \in \mathcal{N}(v) \setminus \mathcal{V}_k$. This prevents local nodes from aggregating a complete neighborhood representation, leading to the information gap illustrated by the cross-client edges in Figure 1 and causing a significant degradation in GNN performance.

We address this missing neighbor problem by reformulating it as a **feature imputation task**. Our proposed method, FedProp, enables each client to locally and efficiently estimate the unknown features of its remote neighbors. This enables the reconstruction of a complete 1-hop neighborhood for message passing, allowing for full-graph GNN updates without violating privacy constraints. The process is illustrated in Figure 2.

To achieve this while strictly preserving privacy, each client constructs an L -hop accessible subgraph $G_k^{(L)} = (V_k^{(L)}, E_k)$. Here, $V_k^{(L)}$ includes all nodes within the L hops of the local nodes of the client k V_k , but the edge set E_k is restricted to only local edges within V_k ¹. Consequently, the remote nodes in $V_k^{(L)}$ appear as isolated leaves, connected solely through their (unknown) boundary edges to

¹Even though we expand the vertex set to $V_k^{(L)}$, the accessible edges remain E_k . In effect, any remote node $u \in V_k^{(L)} \setminus V_k$ appears as an isolated leaf connected only via its (unknown) boundary edge to some local node. This ensures that no new structural information is obtained, preserving the strict privacy constraint

local nodes, to ensure that no new remote structural information is exposed. For practical implementation, we set $L = 1$, which means that clients know the adjacency of their accessible context of one hop, including the boundaries of remote neighbors ($\mathcal{B}_k = \{v \in V_k : \exists u \notin V_k, (v, u) \in E\}$).

Feature Matrix Initialization. For feature propagation, an augmented feature matrix $\mathbf{X}_k^{(0)}$ is constructed for client k ’s accessible subgraph, where known local node features ($\mathbf{x}_i$ for $i \in V_k$) are retained, and features for unknown remote nodes ($i \in V_k^{(L)} \setminus V_k$) are initialized to zeros.

Formulating Imputation as Residual Minimization. Given the accessible subgraph $G_k^{(L)}$ and initialized features $\mathbf{X}_k^{(0)}$, we impute unknown remote node features by minimizing the local Dirichlet residual. For client k ’s accessible subgraph with normalized Laplacian L_k , the local Dirichlet energy is $\mathcal{E}_k(\mathbf{X}_k) = \frac{1}{2} \text{tr}(\mathbf{X}_k^\top L_k \mathbf{X}_k)$. Finding the equilibrium feature matrix $\mathbf{X}_k^*$ such that the gradient of $\mathcal{E}_k$ vanishes on the unknown nodes U_k , i.e., $(L_k \mathbf{X}_k^*)_{U_k} = 0$, effectively driving the energy to a steady state by smoothing their features, corresponds to the steady-state solution of the graph heat equation:

$$\frac{d\mathbf{X}_k(t)}{dt} = -L_k \mathbf{X}_k(t), \quad \text{s.t.} \quad \mathbf{X}_{V_k}(t) = \mathbf{X}_{V_k}^0. \quad (2)$$

The unique analytical solution for the unknown features at this steady state is:

$$\mathbf{X}_{U_k}^* = -L_{U_k U_k}^{-1} L_{U_k V_k} \mathbf{X}_{V_k}.$$

A detailed derivation of this solution is provided in the Appendix (see supplemental materials). While this closed-form solution exists and is unique, directly computing the inverse of $L_{U_k U_k}$ is computationally expensive for large graphs, which motivates our iterative approach.

Imputation as Steady-State of the Heat Equation. Minimizing the Dirichlet residual on the unknown nodes, with fixed boundary conditions on known nodes, corresponds to finding the steady state of the graph heat equation:

$$\frac{d\mathbf{X}_k(t)}{dt} = -L_k \mathbf{X}_k(t), \quad (3)$$

with fixed boundary features $\mathbf{X}_{V_k}(t) = \mathbf{X}_{V_k}^0$ for all t . At steady state, the features no longer change, meaning $\frac{d\mathbf{X}_k}{dt} = 0$, which implies $L_k \mathbf{X}_k^* = 0$. For our constrained problem, this means the Dirichlet residual on the unknown nodes $(L_k \mathbf{X}_k^*)_{U_k}$ vanishes, indicating an equilibrium where feature information has diffused and settled under the fixed known features.

Iterative Solution via Discretization and Boundary Reset. To find the steady state solution, we approximate the solution by discretizing the graph heat equation (Eq. (3)) using the explicit Euler method with a small step size $h > 0$. This yields the iterative update:

$$\mathbf{X}_k^{(t+1)} = \mathbf{X}_k^{(t)} - h L_k \mathbf{X}_k^{(t)}.$$

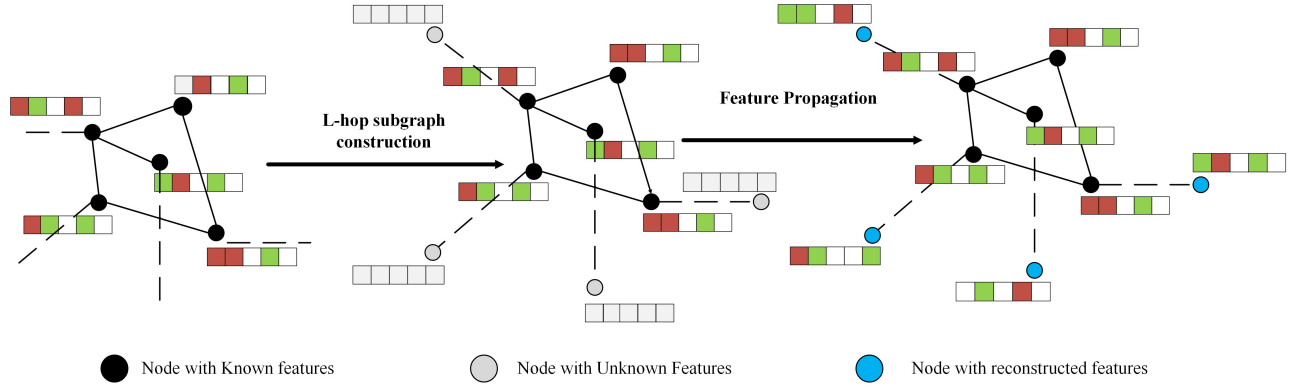


Figure 2: **The FedProp Feature Propagation Process.** The diagram visualizes the FedProp algorithm, which reconstructs missing features for an L-hop subgraph from the initial subgraph. The process involves iteratively propagating features to fill in missing information, allowing for a complete view of the L-hop subgraph.

By setting $P_k := I - hL_k$, this iteration simplifies to $\mathbf{X}_k^{(t+1)} = P_k \mathbf{X}_k^{(t)}$. To enforce the Dirichlet boundary condition that the features of known local nodes ' V_k ' remain fixed at their initial values ' $\mathbf{X}_{V_k}^0$ ', we reset these features at each iteration. The full iterative update with boundary reset is thus:

$$\mathbf{X}_k^{(t+1)} \leftarrow P_k \mathbf{X}_k^{(t)}, \text{ then } \mathbf{X}_k^{(t+1)}(v) = \mathbf{X}_{V_k}^0(v), \forall v \in V_k.$$

To show explicitly how this impacts the unknown nodes, we partition the propagation matrix P_k and the feature matrix $\mathbf{X}_k^{(t)}$ according to known and unknown nodes:

$$P_k = \begin{bmatrix} P_{V_k V_k} & P_{V_k U_k} \\ P_{U_k V_k} & P_{U_k U_k} \end{bmatrix}, \quad \mathbf{X}_k^{(t)} = \begin{bmatrix} \mathbf{X}_{V_k}^0 \\ \mathbf{X}_{U_k}^{(t)} \end{bmatrix}.$$

Applying the update rule and enforcing the fixed ' $\mathbf{X}_{V_k}^0$ ', the iteration for the unknown features $\mathbf{X}_{U_k}$ becomes:

$$\mathbf{X}_{U_k}^{(t+1)} = P_{U_k U_k} \mathbf{X}_{U_k}^{(t)} + P_{U_k V_k} \mathbf{X}_{V_k}^0.$$

Convergence Guarantees. The iterative process for ' $\mathbf{X}_{U_k}^{(t+1)}$ ' is guaranteed to converge linearly to the unique analytical solution ' $\mathbf{X}_{U_k}^*$ ' shown earlier. This convergence is ensured because the spectral radius ' $\rho(P_{U_k U_k})$ ' is strictly less than one, as formally proven in the Appendix (see supplemental materials).

In practice, we terminate the iterative propagation when the Frobenius norm of the change in features between successive iterations falls below a chosen tolerance ε , i.e., ' $\|\mathbf{X}_k^{(t+1)} - \mathbf{X}_k^{(t)}\|_F < \varepsilon$ ', or when the residual itself is sufficiently small, ' $\|L_k \mathbf{X}_k^{(t)}\|_F < \varepsilon$ '. Typically, convergence is achieved within 30–40 iterations on large graphs.

Generalization and other operators The FedProp framework is generalizable to any symmetric propagation operator with a spectral radius bounded by one, which guarantees convergence. In our experiments we use the normalized adjacency matrix ($\hat{A} = D^{-1/2} A D^{-1/2}$), and the diffusion kernel $\exp(-\lambda L)$. Both operators are spectral functions of the graph Laplacian and retain the smoothing properties of

Algorithm 1: Residual-Minimizing Feature Propagation with Boundary Reset

Require: Accessible subgraph $G_k^{(L)} = (V_k^{(L)}, E_k)$, Propagation Matrix $P_k = I - hL_k$, known features $\mathbf{X}_{V_k}^0$, max iterations T , tolerance ε

Ensure: Completed feature matrix $\mathbf{X}_k$

- 1: Initialize $\mathbf{X}_k^{(0)} \leftarrow 0$ on U_k ; set $\mathbf{X}_k^{(0)}(v) \leftarrow \mathbf{X}_{V_k}^0(v)$ for all $v \in V_k$
- 2: **for** $t = 0$ to $T - 1$ **do**
- 3: $\mathbf{X}_k^{(t+1)} \leftarrow P_k \mathbf{X}_k^{(t)}$
- 4: **for all** $v \in V_k$ **do**
- 5: $\mathbf{X}_k^{(t+1)}(v) \leftarrow \mathbf{X}_{V_k}^0(v)$
- 6: **end for**
- 7: **if** $\|\mathbf{X}_k^{(t+1)} - \mathbf{X}_k^{(t)}\|_F < \varepsilon$ **then**
- 8: **break**
- 9: **end if**
- 10: **end for**
- 11: **return** $\mathbf{X}_k^{(t+1)}$

the Laplacian. (We formally outline the sufficient conditions for valid propagation operators in the supplementary material.)

Algorithm. The complete FedProp process is illustrated in Algorithm 1.

Local GNN Training and Advantages. After feature propagation, each client trains a standard GNN (GNN_θ) using the imputed features $\mathbf{X}'_k$ and local adjacency A_k , minimizing cross-entropy loss over labeled nodes $\mathcal{L}_k \subset V_k$. This pre-processing step incurs no additional inter-client communication beyond standard federated model aggregation.

FedProp offers significant advantages:

- **Computational Efficiency:** Costs ' $\mathcal{O}(T_{\text{prop}} \cdot |E_{G_k^{(L)}}| \cdot d)$ ' for sparse matrix-dense matrix multiplication, typically converging in 30–50 iterations without costly inversions.
- **Low Communication:** Matches FedAvg, with communication limited solely to periodic aggregation of model

parameters $\mathcal{O}(R \cdot |\theta|)$. All feature propagation is local.

- **Strong Privacy:** Raw node features and labels remain strictly local; no intermediate representations are transmitted, inherently compatible with Differential Privacy.

A detailed comparison of FedProp’s properties against other federated GNN methods, highlighting these advantages, is presented in the Appendix (see supplemental materials).

5 Experiments and Results

Having proven the convergence of our algorithm, we are now ready to present a detailed experimental evaluation of FedProp in terms of node classification accuracy and communications cost for both IID and non-IID partitions of the underlying graph.

5.1 Experimental Setup

Our experiments use the standard homophilic citation graph datasets: Cora, Citeseer, and Pubmed. Data are partitioned into client subgraphs using a label Dirichlet distribution, with concentration parameter $\beta=10000$ for IID settings and $\beta=10$ or $\beta=1$ for non-IID, to emulate varying degrees of the missing-neighbor challenge.

We demonstrate FedProp’s backbone-agnostic nature by coupling it with two standard GNN architectures: a two-layer **GCN** (Kipf and Welling 2017) and a two-layer **GAT** (Veličković et al. 2018). For feature imputation, we utilize both normalized adjacency (Adj) and discrete diffusion ($Diff$) operators, propagating features for $T = 50$ iterations. We also investigate the impact of enriching features with Random Feature Propagation positional encodings (Eliasof et al. 2023) using $R = 16$ seed vectors propagated for $P = 5$ steps.

We compare FedProp against several reference systems: (i) **Centralized** system serves as a baseline, (ii) **FedProp-Zero** uses subgraph training only (no imputation), and (iii) **FedProp-Full** (oracle) with complete 1-hop remote features (equivalent to sharing all raw features or perfectly reconstructing them). Our method variants are **FedProp (Adj)**, **FedProp (Adj+PE)**, **FedProp (Diffusion)**, and **FedProp (Diffusion+PE)**.

Table 1: Summary of Methods and Information Used

Method	Propagation	Info Used
Centralized GCN	N/A	Full graph
FedProp Zero	N/A	Local subgraph only
FedProp Full	N/A	Full 1-hop subgraph
FedProp (Adj)	Normalized Adjacency	Local + approx. neighbors
FedProp (Adj+PE)	Normalized Adjacency	Local + approx. neighbors + PE
FedProp (Diffusion)	Discrete Diffusion	Local + approx. neighbors
FedProp (Diff+PE)	Discrete Diffusion	Local + approx. neighbors + PE

All hyperparameter details, full dataset statistics, and a more detailed breakdown of baselines are provided in the Appendix (see supplemental materials).

5.2 Overall Node Classification Accuracy

We evaluate the performance of the FedProp framework against several baselines and variants on three standard graph datasets: Cora, Citeseer, and Pubmed. Our experiments, conducted under both Non-IID and IID data partitions, focus on the node classification accuracy of a Graph Convolutional Network (GCN). The results demonstrate that FedProp effectively mitigates the performance degradation caused by missing cross-client connections and achieves near-centralized accuracy.

Table 2 summarizes the mean semi-supervised node classification accuracy and standard deviation for various FedProp variants under a Non-IID data partition. The results clearly show that FedProp achieves highly competitive performance with the centralized GCN baseline and FedProp-Full (the oracle).

Table 2: Node classification accuracy (mean $\pm$ std) under Non-IID partition.

Method	Cora	Citeseer	Pubmed
Centralized GCN	0.811 $\pm$ 0.004	0.700 $\pm$ 0.008	0.785 $\pm$ 0.005
FedProp-Full	0.804 $\pm$ 0.004	0.708 $\pm$ 0.002	0.786 $\pm$ 0.005
FedProp-Zero	0.626 $\pm$ 0.016	0.629 $\pm$ 0.012	0.647 $\pm$ 0.011
FedProp (Adj-PE)	0.748 $\pm$ 0.009	0.663 $\pm$ 0.037	0.773 $\pm$ 0.009
FedProp (Adj)	0.726 $\pm$ 0.005	0.665 $\pm$ 0.007	0.794 $\pm$ 0.004
FedProp (Diff-PE)	0.757 $\pm$ 0.008	0.660 $\pm$ 0.028	0.769 $\pm$ 0.006
FedProp (Diff)	0.733 $\pm$ 0.008	0.661 $\pm$ 0.006	0.792 $\pm$ 0.003

The best FedProp variants for the different datasets and partition settings are summarized in Table 3

Table 3: Best-performing FedProp variant per dataset and partition.

Dataset	Non-IID)	IID
Cora	FedProp (Diff-PE)	FedProp (Diff-PE)
Citeseer	FedProp (Adj-PE)	FedProp (Adj)
Pubmed	FedProp (Adj)	FedProp (Diff)

In contrast, the naive FedProp-Zero baseline, which ignores remote neighbors, shows significantly lower performance across all datasets, confirming the critical impact of missing cross-client connections. For instance, on Cora, FedProp-Zero yields an accuracy of 0.626 ± 0.016 , whereas the other FedProp variants achieve a notable performance gain.

The results of the IID partition, presented in Table 4, show similar trends. The FedProp continues to achieve near-centralized performance, demonstrating its robustness regardless of the data distribution. The naive FedProp-Zero baseline again falls short, reinforcing the value of our feature propagation approach. Interestingly, under the IID par-

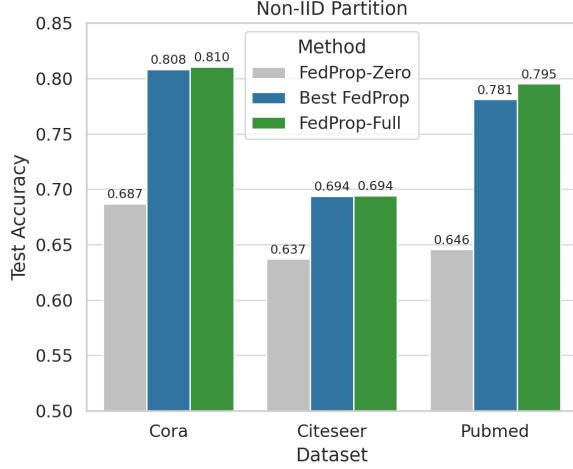


Figure 3: FedProp test accuracy on Cora, Citeseer, and Pubmed Non-IID (bottom) partitions. The best-performing FedProp variant (From Table 3) consistently improves over the Zero-hop baseline and closely matches the oracle (FedProp-Full).



Figure 4: FedProp test accuracy on Cora, Citeseer, and Pubmed under IID partitions.

tion, both FedProp (Adj) and FedProp (Diff) variants show a consistent performance boost.

Table 4: Node classification accuracy (mean $\pm$ std) under IID partition.

Method	Cora	Citeseer	Pubmed
Centralized GCN	0.811 $\pm$ 0.004	0.700 $\pm$ 0.008	0.785 $\pm$ 0.005
FedProp-Full	0.810 $\pm$ 0.004	0.698 $\pm$ 0.006	0.795 $\pm$ 0.004
FedProp-Zero	0.686 $\pm$ 0.008	0.635 $\pm$ 0.007	0.656 $\pm$ 0.009
FedProp (Adj-PE)	0.738 $\pm$ 0.193	0.666 $\pm$ 0.012	0.761 $\pm$ 0.005
FedProp (Adj)	0.790 $\pm$ 0.005	0.689 $\pm$ 0.005	0.780 $\pm$ 0.000
FedProp (Diff-PE)	0.808 $\pm$ 0.007	0.673 $\pm$ 0.017	0.764 $\pm$ 0.005
FedProp (Diff)	0.789 $\pm$ 0.009	0.694 $\pm$ 0.006	0.781 $\pm$ 0.006

We also compare Fedprop to other Federated graph learning approaches. Table 5 reports the node classification accuracy (%) on Cora, Citeseer, and Pubmed for various federated GNN methods under both IID ($\beta = 1$) and non-IID ($\beta = 10000$) partitions with 10 clients. Notably, our proposed FedProp achieves near-centralized performance on Cora under non-IID and 0.800 ± 0.006 under IID—while incurring zero inter-client communication.

It is also noteworthy that FedProp incurs negligible computational overhead (As detailed in the Appendix) and requires no inter-client communication. This makes it highly suitable for constrained federated settings where full neighborhood access is infeasible or expensive. Furthermore, positional encodings significantly improve performance, providing gains in test accuracy, particularly in non-IID scenarios by acting as a structural prior (for details, see the appendix in the supplemental materials).

Table 5: Node classification accuracy on Cora, Citeseer, and Pubmed under federated settings with 10 clients. Baseline results are taken from (Ambekar et al., 2024). Our FedProp method is evaluated under both IID and non-IID partitions.

Method	Cora	Citeseer	Pubmed
GCN (centralized)	0.811 $\pm$ 0.004	0.700 $\pm$ 0.008	0.785 $\pm$ 0.005
DistGAT (non-iid)	0.684 $\pm$ 0.013	0.664 $\pm$ 0.014	0.769 $\pm$ 0.010
DistGAT (iid)	0.645 $\pm$ 0.012	0.635 $\pm$ 0.007	0.745 $\pm$ 0.009
FedGCN (non-iid)	0.778 $\pm$ 0.003	0.680 $\pm$ 0.002	0.772 $\pm$ 0.003
FedGCN (iid)	0.771 $\pm$ 0.003	0.682 $\pm$ 0.008	0.774 $\pm$ 0.002
FedGAT (non-iid)	0.800 $\pm$ 0.005	0.699 $\pm$ 0.004	0.789 $\pm$ 0.004
FedGAT (iid)	0.802 $\pm$ 0.003	0.694 $\pm$ 0.006	0.787 $\pm$ 0.005
FedProp (non-iid)	0.757 $\pm$ 0.008	0.665 $\pm$ 0.009	0.794 $\pm$ 0.003
FedProp (iid)	0.808 $\pm$ 0.006	0.689 $\pm$ 0.010	0.781 $\pm$ 0.005

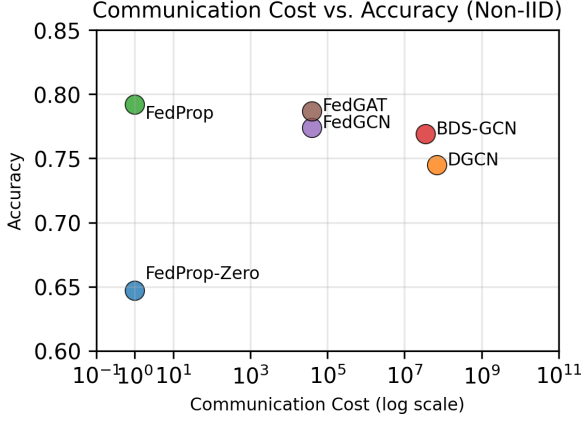


Figure 5: Communication cost vs. accuracy on PubMed. FedProp achieves high accuracy with zero communication.

5.3 Communication Analysis

We analyze the trade-off between communication cost and model performance as measured by the precision across various federated GNN frameworks, focusing on the dataset PUBMED. As shown in Figure 5, **FedProp** achieves superior accuracy with **zero communication cost** incurred during message passing. In other words, our local feature propagation requires no traffic between clients, thereby incurring the same communication cost as when we train with only local subgraph (FedProp-Zero).

FedProp’s communication efficiency compares very favorably against previous methods such as **DGCN** and **BDS-GCN**, which incur very substantial communication overheads due to the need for full or partial neighbor feature exchanges. While other baselines like **FedGCN** and **FedGAT** are more efficient, their requirement for neighbor feature exchange still results in significant communication overheads. By eliminating the need to share neighborhood features, FedProp offers a highly efficient alternative with superior accuracy, making it a particularly appealing solution for tightly constrained federated environments.

5.4 Fedprop Convergence and efficiency

To validate the convergence of our propagation method, we track the reduction in the Dirichlet residual $\|\mathcal{L}X\|_F^2$ over iterations for each client. Figure 6 shows the average delta (change in Dirichlet energy from one iteration to the next) in 10 clients under IID partitioning.

Our iterative feature propagation consistently converges rapidly, with Dirichlet residuals diminishing to near-zero within 40 iterations across all clients. Since we do the propagation steps once before training, the computational overhead is minimal and less than 1% of the overall training time (detailed analysis in the Appendix, see supplemental materials). This confirms that our propagation approach efficiently minimizes the Dirichlet energy residual, yielding a smooth and stable feature estimate constrained by known features.

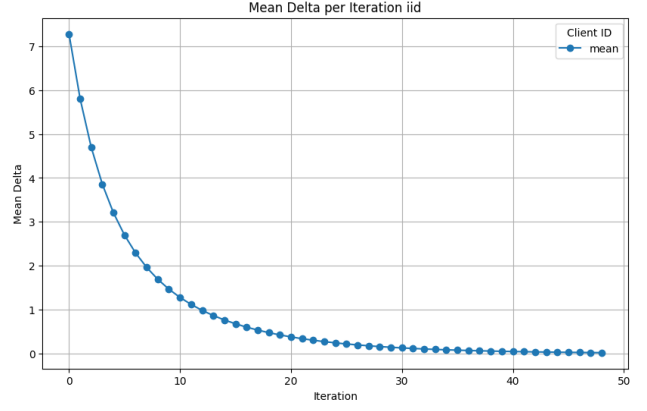


Figure 6: Convergence of the Dirichlet residual $\|\mathcal{L}X\|_F^2$ over iterations under IID partitioning (Cora dataset). The mean residual is averaged over 10 clients.

6 Conclusions and Future Work

We introduced **FedProp**, a communication-free method for federated graph learning that reconstructs missing neighbor features through iterative local propagation. This approach allows each client to train a GNN using only its local graph and initial node features. The fact that FedProp does not require inter-client communication makes it ideal for privacy-sensitive and/or bandwidth-constrained settings.

Our empirical results show that **FedProp**, in particular the **Diffusion-PE** variant, consistently narrows the gap to centralized GNN performance across Cora, Citeseer, and Pubmed, even under challenging non-IID partitions. In particular, Fedprop delivers up to 17% relative improvement over the zero-hop baseline with minimal overhead ($<0.1\%$ increase in runtime), and often matches communication-heavy baselines like FedGCN and FedGAT.

FedProp also achieves the best trade-off between communication cost and test accuracy, as shown in Figure 5. This demonstrates its practical relevance for real-world applications where neighbor access is constrained, including decentralized social graphs, healthcare networks, and edge-device recommendation systems.

7 Limitations and Future Work

Our current work is limited to homophilic graphs and focuses on node classification. Future work could therefore extend FedProp’s capabilities to heterophilic graphs.

Current work also focuses on analytically determined propagation strategies that are derivatives of the graph laplacian; future work could focus on learnable propagation operators.

This would further expand the utility of FedProp as a lightweight, theoretically proven, and empirically validated approach to federated GNN training, with strong promise for scalable and privacy-respecting deployment.

References

Ambekar, S.; Yao, Y.; Li, R.; and Joe-Wong, C. 2024. FedGAT: A Privacy-Preserving Federated Ap-

- proximation Algorithm for Graph Attention Networks. ArXiv:2412.16144 [cs].
- Banabilah, S.; Aloqaily, M.; Alsayed, E.; Malik, N.; and Jararweh, Y. 2022. Federated learning review: Fundamentals, enabling technologies, and future applications. *Information Processing & Management*, 59(6): 103061.
- Cui, Y.; Han, X.; Chen, J.; Zhang, X.; Yang, J.; and Zhang, X. 2025. FraudGNN-RL: A Graph Neural Network With Reinforcement Learning for Adaptive Financial Fraud Detection. *IEEE Open Journal of the Computer Society*, 6: 426–437.
- Eliasof, M.; Frasca, F.; Bevilacqua, B.; Treister, E.; Chechik, G.; and Maron, H. 2023. Graph Positional Encoding via Random Feature Propagation. ArXiv:2303.02918 [cs].
- Fu, X.; Zhang, B.; Dong, Y.; Chen, C.; and Li, J. 2022. Federated Graph Machine Learning: A Survey of Concepts, Techniques, and Applications. ArXiv:2207.11812.
- Gasteiger, J.; Bojchevski, A.; and Günnemann, S. 2022. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. ArXiv:1810.05997 [cs].
- Hamilton, W. L.; Ying, R.; and Leskovec, J. 2018. Inductive Representation Learning on Large Graphs. ArXiv:1706.02216 [cs].
- He, C.; Balasubramanian, K.; Ceyani, E.; Yang, C.; Xie, H.; Sun, L.; He, L.; Yang, L.; Yu, P. S.; Rong, Y.; Zhao, P.; Huang, J.; Annavaram, M.; and Avestimehr, S. 2021. Fed-GraphNN: A Federated Learning System and Benchmark for Graph Neural Networks. ArXiv:2104.07145 [cs].
- Khemani, B.; Patil, S.; Kotecha, K.; and Tanwar, S. 2024. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, 11(1): 18.
- Kipf, T. N.; and Welling, M. 2017. Semi-Supervised Classification with Graph Convolutional Networks. ArXiv:1609.02907 [cs].
- Li, T.; Sahu, A. K.; Talwalkar, A.; and Smith, V. 2020. Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Processing Magazine*, 37(3): 50–60.
- Li, Z.; Bilal, M.; Xu, X.; Jiang, J.; and Cui, Y. 2023. Federated Learning-Based Cross-Enterprise Recommendation With Graph Neural Networks. *IEEE Transactions on Industrial Informatics*, 19: 673–682.
- Liu, R.; Xing, P.; Deng, Z.; Li, A.; Guan, C.; and Yu, H. 2022. Federated Graph Neural Networks: Overview, Techniques and Challenges. ArXiv:2202.07256.
- McMahan, H. B.; Moore, E.; Ramage, D.; Hampson, S.; and Arcas, B. A. y. 2023. Communication-Efficient Learning of Deep Networks from Decentralized Data. ArXiv:1602.05629 [cs].
- Peng, L.; Wang, N.; Dvornek, N.; Zhu, X.; and Li, X. 2023. FedNI: Federated Graph Learning With Network Inpainting for Population-Based Disease Prediction. *IEEE Transactions on Medical Imaging*, 42(7): 2032–2043.
- Reddi, S.; Charles, Z.; Zaheer, M.; Garrett, Z.; Rush, K.; Konečný, J.; Kumar, S.; and McMahan, H. B. 2021. Adaptive Federated Optimization. ArXiv:2003.00295 [cs].
- Rossi, E.; Kenlay, H.; Gorinova, M. I.; Chamberlain, B. P.; Dong, X.; and Bronstein, M. 2022. On the Unreasonable Effectiveness of Feature propagation in Learning on Graphs with Missing Node Features. ArXiv:2111.12128.
- Silva, P. R.; Vinagre, J.; and Gama, J. 2023. Towards federated learning: An overview of methods and applications. *WIREs Data Mining and Knowledge Discovery*, 13(2): e1486. eprint: <https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/widm.1486>.
- Sun, H.; Tu, Z.; Sui, D.; Zhang, B.; and Xu, X. 2025. A Federated Social Recommendation Approach with Enhanced Hypergraph Neural Network. *ACM Trans. Intell. Syst. Technol.*, 16(1): 13:1–13:23.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. ArXiv:1710.10903 [stat].
- Yao, Y.; Jin, W.; Ravi, S.; and Joe-Wong, C. 2023. FedGCN: Convergence-Communication Tradeoffs in Federated Training of Graph Convolutional Networks. ArXiv:2201.12433.
- Zhang, K.; Yang, C.; Li, X.; Sun, L.; and Yiu, S. M. 2021. Subgraph Federated Learning with Missing Neighbor Generation. ArXiv:2106.13430 [cs].
- Zhou, K.; Huang, X.; Zha, D.; Chen, R.; Li, L.; Choi, S.-H.; and Hu, X. 2021. Dirichlet Energy Constrained Learning for Deep Graph Neural Networks. ArXiv:2107.02392 [cs].

A Analytical Solution and Convergence of Feature Propagation

In this section we show that FedProp’s boundary-reset propagation converges to the exact solution of a *Dirichlet feature-imputation* problem on each client’s local subgraph. We first derive the closed-form minimizer and then show that the iterative propagator is a strict contraction that reaches that minimizer.

Setting. Client k observes the undirected subgraph $G_k^{(L)} = (V_k \cup U_k, E_k)$, where V_k are *boundary* nodes with known features $X_{V_k} \in \mathbb{R}^{|V_k| \times d}$ and U_k are *interior* nodes whose features $X_{U_k} \in \mathbb{R}^{|U_k| \times d}$ must be imputed. Let L_k be either the combinatorial or normalized Laplacian on $G_k^{(L)}$ and block-partitioned as follows:

$$L_k = \begin{bmatrix} L_{V_k V_k} & L_{V_k U_k} \\ L_{U_k V_k} & L_{U_k U_k} \end{bmatrix}, \quad X = \begin{bmatrix} X_{V_k} \\ X_{U_k} \end{bmatrix}.$$

Dirichlet SPD condition. Assume every connected component of $G_k^{(L)}$ touches V_k . Then $A := L_{U_k U_k}$ is *symmetric positive definite* (SPD): $A^\top = A$ and $x^\top A x > 0$ for all $x \neq 0$. Anchoring removes the Laplacian’s nullspace, so A is invertible and $0 < \lambda_{\min}(A) \leq \lambda_{\max}(A)$.

A.1 Closed-Form Dirichlet Minimizer

We derive the optimal features X_{U_k} by minimizing the boundary-pinned Dirichlet energy. The objective, with the boundary features X_{V_k} held constant, is to minimize the following residual:

$$\mathcal{R}_k(X_{U_k}) := \frac{1}{2} \|L_{U_k V_k} X_{V_k} + L_{U_k U_k} X_{U_k}\|_F^2.$$

For notational clarity, let $A := L_{U_k U_k}$ and $B := L_{U_k V_k} X_{V_k}$, which allows us to rewrite the objective as $\mathcal{R}_k(X_{U_k}) = \frac{1}{2} \|A X_{U_k} + B\|_F^2$. To find the minimum, we set the gradient with respect to X_{U_k} to zero. Leveraging the symmetry of A , the gradient is:

$$\nabla_{X_{U_k}} \mathcal{R}_k = A^\top (A X_{U_k} + B) = A(A X_{U_k} + B) = 0.$$

Since A is invertible (a consequence of the SPD condition), we can solve for X_{U_k} , yielding the unique minimizer:

$$X_{U_k}^* = -A^{-1}B = -L_{U_k U_k}^{-1} L_{U_k V_k} X_{V_k}.$$

The uniqueness of this solution is guaranteed by the strict convexity of the objective, confirmed by its Hessian $A^\top A \succ 0$. This closed-form solution is precisely the fixed point to which our iterative propagation converges, as we will demonstrate next.

A.2 Propagation Converges to the Analytical Solution

We now show that FedProp’s boundary-reset iteration converges *linearly* to the closed-form minimizer in Eq. (A.1).

The iteration uses the propagator $P := I - hL_k$ for a chosen step size $h > 0$. At each round t we first update all features, $X^{(t+1)} \leftarrow P X^{(t)}$, and then re-clamp the boundary

to its ground-truth values, $X_{V_k}^{(t+1)} \leftarrow X_{V_k}^{(0)}$. For the interior nodes this produces the affine recursion

$$X_{U_k}^{(t+1)} = P_{U_k U_k} X_{U_k}^{(t)} + P_{U_k V_k} X_{V_k}^{(0)}, \quad (4)$$

whose convergence hinges on the linear part $P_{U_k U_k}$.

Contraction of the interior block. Let $A := L_{U_k U_k}$ (SPD), so $P_{U_k U_k} = I - hA$. If λ_i are the eigenvalues of A , those of $P_{U_k U_k}$ are $\mu_i = 1 - h\lambda_i$. Strict contraction, $\|P_{U_k U_k}\|_2 < 1$, requires $|\mu_i| < 1$ for all i , i.e.

$$0 < h < \frac{2}{\lambda_{\max}(A)}.$$

Because $\lambda_{\max}(A) \leq \lambda_{\max}(L_k)$, safe choices are easy: for the normalized Laplacian $0 < h < 1$ suffices, while for the combinatorial Laplacian $0 < h < 1/d_{\max}$ is adequate.

Fixed point via Banach. With $P_{U_k U_k}$ a strict contraction, the Banach fixed-point theorem implies that Eq. (4) converges linearly to the unique $X_{U_k}^*$ solving

$$(I - P_{U_k U_k}) X_{U_k}^* = P_{U_k V_k} X_{V_k}^{(0)}.$$

Substituting $P_{U_k U_k} = I - hL_{U_k U_k}$ and $P_{U_k V_k} = -hL_{U_k V_k}$ gives

$$\begin{aligned} hL_{U_k U_k} X_{U_k}^* &= -hL_{U_k V_k} X_{V_k}^{(0)} \\ \implies X_{U_k}^* &= -L_{U_k U_k}^{-1} L_{U_k V_k} X_{V_k}^{(0)}. \end{aligned}$$

Hence Eq. (A.1) and the fixed point coincide, completing the proof of convergence. In practice the contraction is strong: 30–50 iterations suffice in all experiments.

A.3 Generalisation to Other Symmetric Propagators

The boundary-reset argument is not limited to the linear Laplacian. For any symmetric propagator of the form $P = g(L_k)$ the iteration converges once the interior block is a strict contraction:

$$\rho(P_{U_k U_k}) < 1 \implies X_{U_k}^{(t)} \longrightarrow X_{U_k}^* = (I - P_{U_k U_k})^{-1} P_{U_k V_k} X_{V_k}^{(0)}.$$

Diffusion kernel. For the heat kernel $P_\tau = e^{-\tau L_k}$ the full-graph spectral radius is $\rho(P_\tau) = 1$. Inside FedProp, however, we update only the interior nodes U_k while keeping the boundary nodes V_k fixed. Because the steady-state “all-ones” vector touches the boundary, it cannot live entirely inside U_k . Removing that direction makes the interior block a strict contraction:

$$\rho((P_\tau)_{U_k U_k}) < 1, \quad X_{U_k}^{(t)} \longrightarrow X_{U_k}^*.$$

As $\tau \rightarrow 0$ this fixed point smoothly approaches the Dirichlet solution $-L_{U_k U_k}^{-1} L_{U_k V_k} X_{V_k}^{(0)}$.

Normalised adjacency. For $\hat{A} = D^{-1/2}(A_{\text{raw}} + I)D^{-1/2}$ the largest eigenvalue is again 1, but its eigenvector—the degree-weighted constant vector—also touches V_k . Consequently $\rho(\hat{A}_{U_k U_k}) < 1$ and the iteration converges to

$$X_{U_k}^* = (I - \hat{A}_{U_k U_k})^{-1} \hat{A}_{U_k V_k} X_{V_k}^{(0)}.$$

Any linear blend $P = (1 - h)I + h\hat{A}$ ($h > 0$) inherits the same fixed point because $I - P_{U_k U_k} = h(I - \hat{A}_{U_k U_k})$.

Linear Laplacian (recap). With $P = I - hL_k$ the interior block $P_{U_k U_k} = I - hL_{U_k U_k}$ is a contraction for $0 < h < 2/\lambda_{\max}(L_{U_k U_k})$, and its fixed point is $-L_{U_k U_k}^{-1} L_{U_k V_k} X_{V_k}^{(0)}$, identical to the Dirichlet solution for every admissible step size.

Summary. Anchoring the boundary nodes strips away the non-contracting constant mode from the interior dynamics, turning $P_{U_k U_k}$ into a contraction even when the full propagator has spectral radius 1. The boundary-reset scheme therefore converges linearly for a wide family of propagators, while the linear Laplacian retains the special property of matching the exact Dirichlet minimiser.

B Error Bound for Feature Propagation

This appendix derives a three-term upper bound on the feature reconstruction error. We demonstrate that two seemingly distinct processes—a simplified **global** propagation on the full graph and the practical **client-level** boundary-reset algorithm—can be analyzed through the single, unifying lens of an *affine iterative process*. This unified view reveals that the sources of error are identical in both cases: a transient *propagation error*, a persistent *bias* from the initial conditions, and a fundamental *irreducible error* from graph heterophily.

B.1 The Unified Framework: Affine Iterations

Both propagation dynamics can be described by the general form of an affine iteration:

$$X^{(t+1)} = C X^{(t)} + b, \quad (\text{D.1})$$

where C is an operator that must be a strict contraction ($\|C\|_2 < 1$) and b is a constant bias term. By the Banach fixed-point theorem, any such process is guaranteed to converge to the unique fixed point $X^* = (I - C)^{-1}b$.

To analyze the total reconstruction error, $\|X^{(t)} - X_{\text{true}}\|_F$, our strategy is to introduce the fixed point X^* as an intermediate waypoint. The triangle inequality then splits the total error into two components that we can analyze separately:

$$\|X^{(t)} - X_{\text{true}}\|_F \leq \underbrace{\|X^{(t)} - X^*\|_F}_{(1) \text{ Propagation Error}} + \underbrace{\|X^* - X_{\text{true}}\|_F}_{(2) \text{ Stationary Error}}. \quad (\text{D.2})$$

B.2 Case 1: Global Propagation Analysis

We first analyze the simplified case of propagation on the full graph, where $X^{(t+1)} = PX^{(t)}$. Here, P is a symmetric propagator with a unique top eigenvalue $\lambda_1 = 1$ and spectral gap $\lambda_{\text{gap}} = \max_{i \geq 2} |\lambda_i| < 1$. The fixed point of this process is the projection onto the stationary subspace: $X^* = P_1 X^{(0)}$, where $P_1 = v_1 v_1^\top$. To map this to our affine template, we define the operators:

$$C := P - P_1, \quad b := P_1 X^{(0)}.$$

(1) Propagation Error. Unrolling the error recursion $E_{t+1} = CE_t$ gives:

$$\|X^{(t)} - X^*\|_F \leq \|C\|_2^t \|X^{(0)} - X^*\|_F = \lambda_{\text{gap}}^t \|(I - P_1)X^{(0)}\|_F. \quad (\text{D.3a})$$

(2) Stationary Error. Introducing $P_1 X_{\text{true}}$ as an intermediate point gives:

$$\|X^* - X_{\text{true}}\|_F \leq \|P_1(X^{(0)} - X_{\text{true}})\|_F + \|(I - P_1)X_{\text{true}}\|_F. \quad (\text{D.3b})$$

The Complete Global Bound. Combining the propagation and stationary terms via the split in Eq. (D.2) yields the final three-term bound for the global setting:

$$\boxed{\|X^{(t)} - X_{\text{true}}\|_F \leq \lambda_{\text{gap}}^t \|(I - P_1)X^{(0)}\|_F + \|P_1(X^{(0)} - X_{\text{true}})\|_F + \|(I - P_1)X_{\text{true}}\|_F.} \quad (\text{D.3c})$$

B.3 Case 2: Client-Level Boundary-Reset Analysis

Now we analyze the practical algorithm for a client k , where the update rule for the interior nodes U_k is already in the affine form $X_{U_k}^{(t+1)} = P_{UU} X_{U_k}^{(t)} + P_{UV} X_{V_k}^{(0)}$.

Setup in the Affine Framework. The operators are defined directly as:

$$C := P_{UU}, \quad b := P_{UV} X_{V_k}^{(0)}, \quad \text{and} \quad X_{U_k}^* := (I - P_{UU})^{-1} P_{UV} X_{V_k}^{(0)}.$$

(1) Propagation Error. The rate is governed by $\lambda_{\text{gap}}^{(U)} := \|P_{UU}\|_2 < 1$.

$$\|X_{U_k}^{(t)} - X_{U_k}^*\|_F \leq (\lambda_{\text{gap}}^{(U)})^t \|X_{U_k}^{(0)} - X_{U_k}^*\|_F. \quad (\text{D.4a})$$

(2) Stationary Error. We introduce the ideal reconstruction—the harmonic extension of the *true* boundary features:

$$\|X_{U_k}^* - X_{\text{true}, U_k}\|_F \leq \|(I - P_{UU})^{-1} P_{UV} (X_{V_k}^{(0)} - X_{\text{true}, V_k})\|_F + \|(I - P_{UU})^{-1} P_{UV} X_{\text{true}, V_k} - X_{\text{true}, U_k}\|_F. \quad (\text{D.4b})$$

The Complete Client-Level Bound. Combining these terms gives the final, practical error bound for FedProp:

$$\boxed{\|X_{U_k}^{(t)} - X_{\text{true}, U_k}\|_F \leq \underbrace{(\lambda_{\text{gap}}^{(U)})^t \|X_{U_k}^{(0)} - X_{U_k}^*\|_F}_{\text{Propagation Convergence}} + \underbrace{\|(I - P_{UU})^{-1} P_{UV} (X_{V_k}^{(0)} - X_{\text{true}, V_k})\|_F}_{\text{Initial Boundary Bias}} + \underbrace{\|(I - P_{UU})^{-1} P_{UV} X_{\text{true}, V_k} - X_{\text{true}, U_k}\|_F}_{\text{Irreducible Harmonic Error}}.} \quad (\text{D.5})$$

B.4 Interpretation and Applicability

The unified analysis reveals that the reconstruction error is always governed by the same three fundamental factors:

- **Propagation Convergence** (Term 1): This transient error decays exponentially. In practice, 30–50 iterations are typically sufficient for it to become negligible.

- **Initial/Boundary Bias** (Term 2): This persistent error arises from a low-frequency mismatch between the initial guess (anchored by the boundary) and the truth. It can be mitigated by better initialization strategies (e.g., mean-centering).
- **Irreducible Heterophily** (Term 3): This fundamental error measures the high-frequency signal in the true features that no smooth, harmonic model can perfectly reconstruct. It is a limit of the method, not a flaw.

This client-level analysis is a strict generalization of the global bound; when the boundary set V_k is empty, Eq. (D.5) collapses exactly to the global result in Eq. (D.3c). This theoretical framework applies to any symmetric propagator that satisfies the spectral conditions laid out in Table 6, making the insights broadly applicable across graph learning.

Operator	Built from	Formula (P)	Safe range
Adj/GCN	$\hat{A}$	$\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$	with self-loops
Diffusion	L	$I - hL$	$0 < h < 2/\lambda_{\max}(L)$
APPNP	$\hat{A}$	$(1 - \alpha)I + \alpha\hat{A}$	$0 < \alpha < 1$
Heat kernel	L	e^{-tL}	$t > 0$

Table 6: Common symmetric propagators meeting the spectral conditions.

C Extended Experimental Setup and More Results

C.1 Datasets and Statistics

We use the standard homophilic Planetoid citation graphs: CORA, CITESEER, and PUBMED. Table 7 reports node/edge counts, number of classes, feature dimensionality, and average degree.

Table 7: **Dataset Statistics.** $|V|$: nodes, $|E|$: edges, C : classes, F : feature dimension, d : average degree.

Dataset	$ V $	$ E $	C	F	d
Cora	2,708	5,429	7	1,433	4.01
Citeseer	3,327	4,732	6	3,703	2.84
Pubmed	19,717	44,338	3	500	4.50

C.2 Client Partitioning Protocol

We partition each dataset into K client subgraphs using the *label-Dirichlet* scheme from Yao et al. (2023). For class set $\{1, \dots, C\}$, we draw class weights $\alpha \sim \text{Dir}(\beta \mathbf{1}_C)$ *independently per client*, and assign nodes of class c to client k proportionally to $\alpha_c^{(k)}$. We use:

- **IID**: $\beta=10000$ (nearly uniform across clients),
- **Non-IID**: $\beta=10$ and $\beta=1$ (increasing heterogeneity).

This emulates the *missing-neighbor* challenge seen in cross-silo deployments: smaller β yields stronger label imbalance,

which increases the fraction of cross-client edges and exacerbates incomplete neighborhoods for local message passing.

C.3 Backbones and Optimization

We couple FEDPROP with two standard backbones:

GCN (Kipf and Welling 2017) Two layers, 16 hidden units, ReLU, dropout 0.5 per layer.

GAT (Veličković et al. 2018) Two attention layers, 8 heads $\times$ 8 features (hidden size 64), ELU, dropout 0.6.

Optimization. For CORA/CITESEER we use SGD (lr 0.5, weight decay 5×10^{-4}); for PUBMED, Adam (lr 0.01, same decay). Training runs up to 600 epochs with early stopping (patience 10) on validation loss. All federated experiments employ FEDAVG with synchronous aggregation at each round.

C.4 Feature Propagation Configuration

We reconstruct missing remote-neighbor features locally via iterative propagation on each client’s accessible subgraph. We consider two symmetric propagation operators:

- **Adjacency** (Adj): $\hat{A} = D^{-\frac{1}{2}}(A+I)D^{-\frac{1}{2}}$.
- **Diffusion** ($Diff$): explicit-Euler discretization of the heat equation with the normalized Laplacian $L = I - \hat{A}$, i.e., $X^{(t+1)} \leftarrow (I - hL)X^{(t)}$ with Dirichlet boundary reset on known (local) nodes.

Unless stated otherwise, we use a fixed budget of $T=50$ iterations and step size $h=1$ (stable for the normalized Laplacian). We terminate early if $\|X^{(t+1)} - X^{(t)}\|_F < 10^{-6}$.

C.5 Graph Positional Encoding via Random Feature Propagation

To enrich node features with structural signals, we use *Random Feature Propagation (RFP)* (Eliasof et al. 2023), a lightweight positional encoding method based on diffusing random vectors over the graph.

Let $B^{(0)} \in R^{n \times R}$ be a matrix of R i.i.d. Gaussian vectors (one per node). We propagate these features over P steps using a symmetric operator P and apply row-wise ℓ_2 normalization at each step:

$$B^{(t+1)} \leftarrow \text{row_norm}(P \cdot B^{(t)}), \quad t = 0, \dots, P-1,$$

$$B \leftarrow [B^{(0)} \mid B^{(1)} \mid \dots \mid B^{(P)}] \in R^{n \times R(P+1)}.$$

Defaults. Unless stated otherwise, we set:

$$P = \hat{A} = D^{-1/2}(A + I)D^{-1/2}, \quad R = 16, \quad P = 5.$$

Usage. We concatenate the resulting positional encoding B to the (reconstructed) node features prior to GNN training.

Cost. Computing RFP incurs $O(P|E|R)$ time for sparse-dense multiplications and uses $O(nR(P+1))$ memory.

C.6 Baselines and Variants

We evaluate:

Centralized GNN trained on the full graph with complete features and edges (upper bound).

FedProp-Zero *No imputation*: remote neighbors are featureless; clients still participate in FEDAVG.

FedProp-Full Oracle with exact 1-hop remote features available to each client prior to training.

FedProp (Adj) Feature imputation via $\hat{A}$ for $T=50$ iterations; no PE.

FedProp (Adj+PE) As above, plus RFP encodings.

FedProp (Diffusion) Feature imputation via $(I - hL)$ iterations (heat equation) with boundary reset; no PE.

FedProp (Diffusion+PE) Diffusion imputation with RFP.

C.7 Evaluation Protocol (Reproducibility)

Unless noted, results are averaged over 10 runs with different seeds; we report mean $\pm$ std on the test split. We follow the standard Planetoid splits; hyperparameters are fixed across runs per dataset/setting. *Communication cost* counts only model-parameter exchanges during aggregation; propagation is purely local and incurs no inter-client traffic.

Results with GAT backbone. To further validate the backbone-agnostic design of FedProp, we report node classification results using a Graph Attention Network (GAT). Tables 8 and 9 show that FedProp consistently outperforms the FedProp-Zero baseline and approaches the performance of centralized and full-information models, mirroring the trends observed with GCN backbones.

Table 8: Node classification accuracy (mean $\pm$ std) for FedProp-GAT across 10 clients under **IID** partition

Method	Cora	Citeseer	Pubmed
FedProp-Full	0.792 $\pm$ 0.014	0.682 $\pm$ 0.008	0.752 $\pm$ 0.029
FedProp-Zero	0.526 $\pm$ 0.021	0.583 $\pm$ 0.007	0.558 $\pm$ 0.018
FedProp (Adj-PE)	0.729 $\pm$ 0.024	0.648 $\pm$ 0.013	0.756 $\pm$ 0.031
FedProp (Adj)	0.711 $\pm$ 0.022	0.641 $\pm$ 0.015	0.785 $\pm$ 0.022
FedProp (Diffusion-PE)	0.739 $\pm$ 0.021	0.661 $\pm$ 0.008	0.776 $\pm$ 0.018
FedProp (Diffusion)	0.722 $\pm$ 0.016	0.629 $\pm$ 0.023	0.783 $\pm$ 0.020

Table 9: Node classification accuracy (mean $\pm$ std) for FedProp-GAT across 10 clients under **non-IID** partition

Method	Cora	Citeseer	Pubmed
FedProp-Full	0.784 $\pm$ 0.031	0.686 $\pm$ 0.012	0.767 $\pm$ 0.030
FedProp-Zero	0.506 $\pm$ 0.015	0.599 $\pm$ 0.011	0.595 $\pm$ 0.052
FedProp (Adj-PE)	0.695 $\pm$ 0.019	0.675 $\pm$ 0.008	0.748 $\pm$ 0.019
FedProp (Adj)	0.695 $\pm$ 0.014	0.670 $\pm$ 0.015	0.781 $\pm$ 0.021
FedProp (Diffusion-PE)	0.720 $\pm$ 0.018	0.675 $\pm$ 0.010	0.752 $\pm$ 0.041
FedProp (Diffusion)	0.709 $\pm$ 0.011	0.671 $\pm$ 0.010	0.772 $\pm$ 0.018

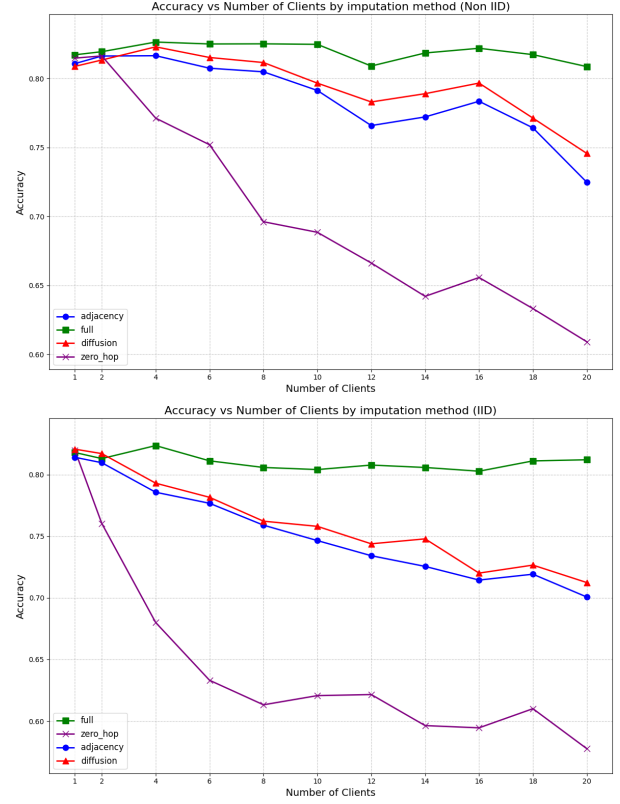


Figure 7: **Scalability with number of clients** on the Cora dataset. Test accuracy under increasing client count for (top) non-IID and (bottom) IID partitions. Zero-hop (no imputation) degrades sharply with fragmentation. In contrast, FedProp with diffusion-based propagation remains stable and closely tracks the oracle (FedProp-Full).

C.8 Scalability with Respect to Number of Clients

To assess how FedProp scales with data fragmentation, we vary the number of clients from 1 to 20 and evaluate test accuracy on the CORA dataset under both IID and non-IID label partitioning schemes. Figure 7 shows the resulting accuracy trends across different imputation strategies.

As expected, increasing the number of clients leads to a general decline in performance for all methods due to reduced local neighborhood visibility and increased fragmentation. However, this decline is particularly steep for the FEDPROP-ZERO baseline, especially in the non-IID regime, where local label bias exacerbates information loss.

In contrast, FedProp with diffusion-based imputation is highly robust: it preserves high accuracy across a wide range of client counts and shows only a modest decline relative to the centralized upper bound. These results demonstrate that local propagation effectively mitigates the challenges posed by extreme data partitioning in federated graph learning.

C.9 Impact of Positional Encoding (PE)

Positional encodings. While FedProp relies on local feature propagation to reconstruct missing neighbor informa-

tion, structural biases can further improve learning in fragmented graphs. We explore whether augmenting node features with positional encodings (PE), derived from random feature propagation (RFP), improves performance under different partition regimes and propagation operators.

Role of positional encoding. In addition to reconstructing missing neighbor features via propagation, we investigate the impact of incorporating structural positional encodings. Specifically, we evaluate whether augmenting node features with random feature propagation (RFP) improves classification accuracy across various propagation settings and partition regimes.

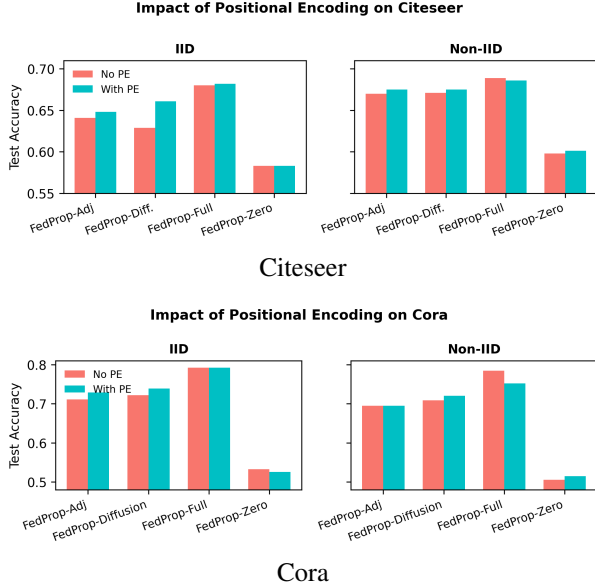


Figure 8: Impact of positional encoding on client accuracy (averaged over 10 clients) under **non-IID partitioning** ($\beta = 1$). Each bar pair compares test accuracy with and without PE for different propagation operators.

Accuracy. Positional encoding (PE) has negligible effect on the zero-hop baseline across both Citeseer and Cora (typically $< 0.3\%$ improvement), as it lacks neighborhood context for propagation. In contrast, both **adjacency-based** and **diffusion-based** propagation variants benefit significantly from PE—gaining up to $+1.8\%$ in accuracy under non-IID settings.

Partition sensitivity. PE has a stronger effect in the non-IID regime ($\beta = 1$), where local structure is highly skewed. Under IID partitions ($\beta = 10000$), clients already receive balanced samples, and the additional inductive signal from PE yields smaller gains.

Recommendation. Given its low computational cost and consistent accuracy improvements in the non-IID setting, we enable PE by default in all remaining experiments unless otherwise stated.

C.10 Privacy Analysis and comparison

Deployment assumption (one hop). Each client is assumed to have local access to its one-hop neighborhood (typical in single-provider settings such as a bank ledger, a social platform’s first-party graph, or enterprise IT logs). In cross-organization settings, access to cross-entity links requires prior data-sharing agreements and governance.

Privacy summary. FedProp is privacy-by-design: raw features, labels, and edges never leave the client; only model parameters/updates are transmitted, matching the communication surface of standard FL. We assume an honest-but-curious server over secure channels; while inference on updates is possible in principle, FedProp is fully compatible with Secure Aggregation and Differential Privacy to obtain formal guarantees.

Compliance and governance. This design follows data-minimization principles and reduces data-movement and re-identification risk, aligning with regulatory frameworks (e.g., GDPR, CCPA) and domain rules (e.g., HIPAA). In many deployments the graph structure is first-party (available to the provider) while node features/labels remain sensitive; FedProp supports such cases without structural exchange, and can be hardened with SA/DP where policy requires formal guarantees.

At-a-glance comparison. Table 10 contrasts what different federated GNN methods share each round, the resulting structural disclosure risk, and per-client overhead.

Table 10: Privacy and overhead across federated GNN methods (per round).

Method	Shared information	Structural disclosure risk	Client overhead
FedProp (ours)	Model parameters only	None (structure kept local)	Low
FedAvg [†]	Model parameters only	N/A	Low (baseline)
FedGCN	Model params + <i>neighbor features</i>	Yes (direct feature exchange)	Moderate
FedGAT	Model params + surrogate metadata	Yes (metadata can leak structure)	Moderate
FedSage+	Model params + <i>synthetic neighbors</i>	Yes (synthetics encode structure)	High
FedNI	Model params + GAN-based features	Yes (GAN outputs encode structure)	High

[†] Non-graph baseline (for reference).